

The method `simulateMachine` should operate the machine based on the list of inputs and return the number of coins and candies left in the machine at the end. For example, if the input Machine has 10 coins and 5 candies, and a total of 4 candies are successfully bought, the output should be `(14, 1)`.

```
def simulateMachine(inputs: List[Input]): State[Machine, (Int, Int)]
```

6.7 Summary

In this chapter, we touched on the subject of how to write purely functional programs that have state. We used random number generation as the motivating example, but the overall pattern comes up in many different domains. The idea is simple: use a pure function that accepts a state as its argument, and it returns the new state alongside its result. Next time you encounter an imperative API that relies on side effects, see if you can provide a purely functional version of it, and use some of the functions we wrote here to make working with it more convenient.

